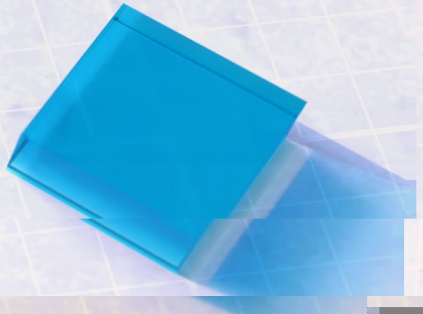
Workshop: Taking back control of your code

using ast-grep & opengrep tools 🦾

Keeping control of code

— is challenging

is challenging 🧗 … once it grows 🌱 … and it ages 🧓



Keeping control of code

— is challenging

is challenging 🧗 … using generative AI 🤖 ➡ AI Slop 🥣

Keeping control of code — is well supported

is well supported:

- 💻 through your IDE
  - ➢ 💡 refactoring, usage search, quick fixes, etc.
  - ➢ 🔌 Language Server Protocol (LSP)
- ⚠️ through type checking
- 🛡️ via linting and QA tooling

Keeping control of code — is problematic

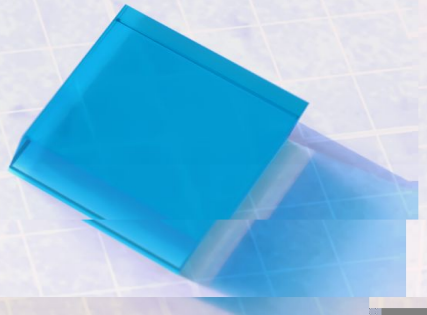
is also problematic:

- 🔍 textual search on code is error prone
  - ➢ 📐 unexpected differences in formatting
  - ➢ 🔣 regex difficult & not always suitable
- ✍️ manual changes are time consuming & error prone
- 🪜 off-the-shelf linting might not suit all your needs

Keeping control of code — Software Maintenance

is all about the 🛠️ 4 Types of Software Maintenance

- 🐞 Corrective Software Maintenance
- 🚫 Preventive Software Maintenance
- 🔄 Adaptive Software Maintenance
- ✨ Perfective Maintenance



Corrective Software Maintenance

Fixing bugs and errors found in your software
... after it has been released.

🐞 Corrective Software Maintenance

🪛 is like patching up holes in a road

😬 but what if …
😢 … a single hole turns out to be lots and lots …
😰 … since you've been doing it all wrong for weeks …

🤔 … but, can't I use search to find my holes in the code

🚫 Preventive Software Maintenance

prevent potential problems before they occur

🚫 Preventive Software Maintenance

- 🧿 Involves regular monitoring and updates
- 🛡️ Static analysis are a great preventative using:
  - ➢ 🪚 Off-the-shelf linting tools and rules
  - ➢ 🕵️ Static Application Security Testing (SAST)
  - ➢ 👨‍💻 Custom linting rules

🔄 Adaptive Software Maintenance

updating software for new or changing environments

🔄 Adaptive Software Maintenance

- e.g. changing code to migrate to:
  - ➢ 📚 more recent (major) version
  - ➢ 🧩 another language / platform API
- can (partially) be automated:
  - ➢ 🦾 off-the-shelf ***and*** custom code rewrites

✨ Perfective Maintenance

improving and enhancing your software… e.g. by:

- 🎁 adding new features
- 🎨 improving user interfaces
- 🚀 optimizing app performance

✨ Perfective Maintenance

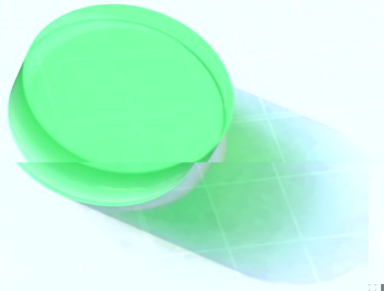
can assisted with (among others):

- 🔦 code insights gained by structural search
- 💡 context aware Quick Fixes in your IDE

Software Maintenance — 🦾 ast-grep & Opengrep

Software Maintenance can be made more easy using …

… 🦾 ast-grep & Opengrep (fork of Semgrep CE / OSS)



ast-grep — swiss army knife for code

is like a swiss army knife for code:

- 🔎 syntax aware (structural) search
- 🛡️ DIY custom linting
  - 🩹 with autofix support
- 🔄 automated code rewrites (codemods)
- 🪄 custom Quick Fixes inside IDE (through LSP)

ast-grep — characteristics

has the following characteristics

- 🌍 polyglot with 20+ languages out of the box
  - ➢ 🔌 but, supports any treesitter grammar
- ⚡ Extremely fast (written in Rust)
- 🤝 interactive mode for (auto) fixes
- 🧩 Node.js, Python an Rust API

ast-grep — embedded languages

supports embedding a language inside another

- known as “multi-language documents”
- practical usages:
  - ➢ 🌐 HTML `<script>` and `<style>` tags
  - ➢ 🪆 CSS-in-JS and HTML-in-JS
  - ➢ 🤔 inline SQL in backend code ❔

ast-grep — disadvantages

also has some disadvantages

- 😜 structural search can be quirky
- 🧗 learning curve advanced AST matching rules
- 😿 unaware of language semantics
  - ➢ 🗒️ syntax variations must be explicit

Opengrep — alternative to ast-grep

is an interesting alternative to ast-grep

- 🌍 polyglot with 30+ languages out of the box
- 🔎 advanced structural search is
  - 🧘 aware of syntax variations (less is more)
  - 💫 all you need to create rules (no AST)
  - 🧬 aware of language specific semantics

➢ aka semantic search

Opengrep — alternative to ast-grep

is an interesting alternative to ast-grep (continued)

- ⏩ autofix support, but not interactively

Opengrep — fork of Semgrep CE

is a fork of Semgrep Consumer Edition (CE) *

- previously also know as Semgrep OSS
- forked by 10+ Security Companies in january 2025
- triggered by Semgrep's changes in:
  - ➢ licensing of its SAST rules
  - ➢ feature removal from Community Edition

* more info in "Announcing OpenGrep" blogpost: https://pulse.latio.tech/p/announcing-opengrep

Opengrep — fork of Semgrep CE

is a fork of Semgrep Consumer Edition (CE) (continued)

- 🔮 might turn out to be better Semgrep CE
  - ➢ consortium of 10+ organizations
  - ➢ fixes long standing bugs
  - ➢ improved performance
  - ➢ (re)introducing features of non-free version

Opengrep — similarities with ast-grep

is quite similar to ast-grep

- 🔌 generic IDE integration (through LSP)
- 🔳 similar CLI usage for structural search

```
ast-grep --pattern .. --lang=..
open-grep --pattern .. --lang=..
```

- 📜 uses similar looking YAML rules *

* LLMs often mistakenly use Opengrep / Semgrep rule syntax when asked for an ast-grep rule

Opengrep — syntax similar to ast-grep

uses syntax similar to ast-grep

- exact same syntax for (single) meta variable

```
C      ✅  a nrspc
       ✅   m  a nrspc
 C     ⚠️  a nrspc  g  Mnc epcn
```

Opengrep — syntax similar to ast-grep

uses syntax similar to ast-grep (continued)

- ellipsis vs non-capturing multi metavariable *

```
Mnc epcn    c epcn AC
  r epcn
```

- ellipsis metavariables vs multi metavariables *

```
 E          Mnc epcn    c epcn AC
E             r epcn
```

* OpenGrep syntax is not restricted by syntax of target language like ast-grep

Opengrep — alternative to ast-grep

is an interesting alternative to ast-grep

- 🕵️ bundled SAST rules
- 🕸️ taint analysis from “source” to “sink”

Opengrep — disadvantages to ast-grep

has some disadvantages compared to ast-grep

- 🐌 slower than ast-grep
- ⚠️ “less-is-more” approach might be less suitable
  - ➢ code rewrite requiring exact matching
- 🧩 no API for advanced usage
- 👮 primary focus is on security scanning

🤺 Opengrep vs ast-grep

which tool(s) is the right one depends on

- 🧑‍🎨 personal preference
- 🤷 my advice (at time of writing)
  - ➢ advanced code rewrites ➡ ast-grep
  - ➢ semantic search ➡ Opengrep
  - ➢ interactively applying autofixes ➡ ast-grep
  - ➢ multi-language documents ➡ ast-grep

Workshop: Taking back control of your code

Use my fork of `home-assistant-frontend` at https://github.com/evangalen/home-assistant-frontend

- ▶️ start reading `01_intro.md` file in `workshop` folder
- ℹ️ also see `99-useful-resources.md` file
- 🌟 and `slidedeck.pdf` file